

DA5402W – Machine Learning Operations Lab

Assignment-3 Submission

**Deploying PyTorch ML Workloads with
Docker & Kubernetes**

Name: BELHE ROHAN SHIRISH

Roll Number: DA25M555

Email: da25m555@smail.iitm.ac.in

Indian Institute of Technology Madras

Contents

1	Assignment 3 Overview	2
2	Implemented Solution	2
2.1	Repository and Git Workflow	2
2.2	PyTorch Training and Model Serving	3
2.3	Docker and Kubernetes	3
3	Submission Links	3
4	End-to-End Validation Summary	4
5	What I Found Most Challenging	4
6	AI-Assistance Disclosure	5

1 Assignment 3 Overview

The objective of Assignment 3 is to take a PyTorch image-classification model through a complete deployment lifecycle: structured local development using Git, containerized training and serving with Docker, and orchestrated execution on Kubernetes. The resulting system is a production-style pipeline capable of training a model, persisting its checkpoint, exposing predictions through an API, and scaling the serving workload.

The assignment includes the following major requirements:

1. Create a public GitHub repository named `mlops-pytorch-pipeline` with `main`, `develop`, and feature branches. All feature work must be merged through meaningful pull requests using Conventional Commit messages.
2. Implement a PyTorch CNN or fine-tuned torchvision model for CIFAR-10 or Fashion-MNIST, including DataLoaders, transforms, YAML-based hyperparameters, JSON-line metrics, checkpoint saving, and early stopping.
3. Implement a Flask or FastAPI serving application with `POST /predict` and `GET /health` endpoints.
4. Build separate optimized Docker images for training and serving. The serving image must use a slim base image, expose port 8080, run as a non-root user, and contain a Docker `HEALTHCHECK`.
5. Deploy the training workload as a Kubernetes Job with a ConfigMap, persistent dataset and checkpoint storage, and resource requests and limits.
6. Deploy the serving workload with two replicas, readiness and liveness probes, rolling updates, a ClusterIP Service, and a CPU-based Horizontal Pod Autoscaler.
7. Demonstrate an end-to-end run and record successful training, deployment, health-check, autoscaling, and prediction evidence in the final pull request.

2 Implemented Solution

2.1 Repository and Git Workflow

The project was developed in a public GitHub repository using a dedicated `develop` branch and multiple feature branches. The model, API, Docker, Kubernetes, CI, documentation, and final rubric-alignment changes were reviewed and merged through pull requests. Conventional Commit messages were used to keep the project history clear and auditable.

2.2 PyTorch Training and Model Serving

A Fashion-MNIST convolutional neural network was implemented using PyTorch. The training program reads its hyperparameters from `configs/training_config.yaml`, emits structured JSON-line metrics, supports early stopping, and saves the best model to `classifier_v1.pt`. A FastAPI service loads this checkpoint and exposes:

- GET `/health` to confirm successful model loading; and
- POST `/predict` to accept an image and return the predicted class, class index, and probabilities for all ten Fashion-MNIST classes.

2.3 Docker and Kubernetes

Separate Dockerfiles were created for training and serving. The training image uses a multi-stage structure, while the serving image uses a slim Python base, runs as a non-root user, exposes port 8080, and includes a health check.

The Kubernetes implementation uses the `ml-training` namespace, a ConfigMap mounted at `/app/configs`, a 2 Gi dataset PVC, and a 1 Gi checkpoint PVC. The training Job requests and limits 2 CPU cores and 4 Gi of memory. The serving Deployment runs two replicas with a rolling-update strategy, health probes, resource controls, a ClusterIP Service on port 80 targeting port 8080, and an HPA configured for two to four replicas at 70% target CPU utilization.

3 Submission Links

Public GitHub Repository	https://github.com/belherohan-iitmda25m555/mlops-pytorch-pipeline
Final PR with End-to-End Logs	https://github.com/belherohan-iitmda25m555/mlops-pytorch-pipeline/pull/9

The final pull request description records the complete validation results, including unit tests, Docker checks, containerized training and inference, Kubernetes dry-run validation, Job completion, bound PVCs, two healthy serving replicas, Service and HPA status, and successful health and prediction requests.

4 End-to-End Validation Summary

Validation Item	Result
Unit tests	2 tests passed
Dockerfile checks	Completed with no warnings
Docker training	2 epochs completed
Validation accuracy	89.94%
Checkpoint	classifier_v1.pt saved
Container health and prediction	Both successful; HTTP 200
Kubernetes dry-run	All 8 resources validated
Kubernetes training Job	Complete, 1/1
Training data PVC	Bound, 2 Gi
Checkpoint PVC	Bound, 1 Gi
Serving Deployment	2/2 replicas available, zero restarts
ClusterIP Service	Port 80 targeting container port 8080
Horizontal Pod Autoscaler	CPU metrics available; 2–4 replicas
Kubernetes health and prediction	Both successful; HTTP 200

5 What I Found Most Challenging

The most challenging part of this assignment was integrating the Dockerized training and serving workloads with Kubernetes while keeping the implementation faithful to the required deployment specification. Each component worked independently, but the complete workflow introduced dependencies between image availability, configuration, storage, resource allocation, model creation, and API startup.

One important challenge was understanding that Docker Desktop and Minikube maintain separate image stores. Even though both images had been built and tested locally, the Kubernetes pods could not use them until the images were explicitly loaded into Minikube. This made `imagePullPolicy` and local image verification important parts of the debugging process. The PyTorch images were also large, so builds and image transfers took several minutes and sometimes appeared inactive despite progressing normally.

Persistent storage required careful sequencing. The training Job needed one PVC for

the Fashion-MNIST dataset and another for the generated classifier checkpoint. The ConfigMap had to be mounted at the expected configuration path, and the checkpoint produced by the completed training Job then had to be readable by both serving replicas. A deployment started before the checkpoint existed would not become healthy, so I validated the system in stages: namespace and storage, training completion, serving deployment, Service, HPA, and finally the endpoints.

I also encountered a source-level issue when accidental terminal command text was appended to `dataset.py`. The Docker container exposed this as a Python syntax error. I used the traceback to identify the exact line, corrected it, rebuilt the image, and repeated the training and serving tests. This reinforced why automated syntax checks and unit tests belong in CI rather than relying only on manual execution.

The final challenge was aligning a functionally successful deployment with the exact rubric. I revised the namespace, persistent volumes, replica count, probe timings, rolling-update strategy, resource requests and limits, Service ports, and HPA range. I then repeated dry-run validation and the full Kubernetes workflow. The corrected Job completed successfully, two serving pods became healthy with zero restarts, CPU metrics reached the HPA, and both health and prediction requests returned successful responses. This process demonstrated that MLOps quality depends not only on model accuracy, but also on reproducibility, observability, infrastructure correctness, and evidence-based validation.

6 AI-Assistance Disclosure

OpenAI ChatGPT was used as a learning and review assistant for project planning, command guidance, debugging, document preparation, and rubric verification. All suggestions were reviewed, tested, adapted, and validated by the author. The author is responsible for the final implementation and is prepared to explain every component during a code review.